

DATA STRUCTURES

Python Provides several built-in data structures that help us organizing and manage data efficiently.

List

Tuple

set

dict

str

List in Python

1. What is List?

A list is a collection of items that are ordered, mutable (changeable) and allow duplicate elements. List can hold items of different data types, such as integers, or even other lists.

Syntax:

```
my_list = [element 1, element 2, element 3, ...]
```

Example:

```
Fruits = ["apple", "banana", "cherry"]
```

```
number = [1, 2, 3, 4, 5]
```

```
mixed = ["apple", 3, True]
```

2. Accessing List Elements

You can access individual elements in a list using indexing. Remember that Python uses zero-based indexing, meaning the first item is at index 0.

(Syntax:

list-name [index]

Example/:

fruits = ["apple", "banana", "cherry"]

print (fruits[0]) # output: apple

print (fruits[2]) # output: cherry

you can also use negative indexing to access elements from the end of the list:

print (fruits[-1]) # output: cherry

print (fruits[-2]) # output: banana

(③ Modifying Lists

List are mutable, which means you can change the value of items in a list

changing a specific element

fruits[1] = "orange"

print (fruits) # output: ['apple', 'orange', 'cherry']

Adding elements:

• append(): Adds an element to the end of the list

fruits.append("grape")

print (fruits) # output: ['apple', 'orange', 'cherry', 'grape']

• `insert()`: Inserts an element at a specific index
`fruits.insert(1, "Kiwi")`
`print(fruits)` # output: ['apple', 'Kiwi', 'orange', 'cherry']

Removing elements:

• `remove()`: Removes the first occurrence of an element

`fruits.remove("orange")`
`print(fruits)` # output: ['apple', 'Kiwi', 'cherry']

• `pop()`: ~~Removes~~ Removes the element at a specific index
or the last item if no index is provided.

`fruits.pop()` # Removes the last item
`print(fruits)` # output: ['apple', 'Kiwi']

`fruits.pop(0)` # Removes the first item
`print(fruits)` # output: ['Kiwi']

• `clear()`: Removes all elements from the list

`fruits.clear()`
`print(fruits)` # output: []

④ Slicing Lists:

You can extract a portion of a list using slicing

Syntax:

`list_name[start : stop : step]`

- start: The index to start the slice (inclusive)
- stop: The index to stop the slice (exclusive)
- step: The number of steps to skip elements (default: 1)

Examples:

numbers = [0, 1, 2, 3, 4, 5, 6]

Print (numbers[1:4]) # output: [1, 2, 3] (from index 1 to 3)

Print (numbers[:4]) # output: [0, 1, 2, 3] (from start to index 3)

Print (numbers[2:]) # output: [2, 3, 4, 5, 6] (from index 2 to end)

Print (numbers[::2]) # output: [0, 2, 4, 6] (every 2nd element)

5. List Functions and methods

Python provides several built-in functions and methods for working with lists.

Common Function

- len(list): Returns the number of elements in the list

Print (len(fruits)) # output 3

- Sorted(list): Returns a new sorted list without changing the original list.

numbers = [5, 2, 9, 1]

Print (Sorted(numbers)) # output: [1, 2, 5, 9]

print (numbers) # original list remains unchanged: [5, 2, 9, 1]

- Sum(list): Returns the sum of elements in a list (for numerical lists)

numbers = [1, 2, 3, 4]

Print (sum(number)) # output: 10

Common methods:

- `index(element)`: Returns the index of the first occurrence of the specified element.
`print( fruits, index("apple"))` # output : 0
- `count(element)`: Returns the number of occurrences of an element in the list.

`numbers = [1, 2, 3, 1, 1]`

`print( numbers, count(1))` # output : 3

- `reverse()`: Reverses the element of the list in place.

`fruits.reverse()`

`print( fruits)` # output : ['cherry', 'orange', 'apple']

- `sort()`: Sorts the list in place (ascending by default)

`numbers = [5, 2, 9, 1]`

`numbers.sort()`

`print( numbers)` # output : [1, 2, 5, 9]

⑥ Nested Lists

List can contain other lists, allowing you to create nested lists. This can be useful for storing matrix-like data structures.

Example:-

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9].  
]
```

Accessing elements in a nested list (first row)
`print( matrix[0])` # output : [1, 2, 3]
`print( matrix[1][1])` output : 5 (element in the second row
Column)

TUPLES AND SETS IN PYTHON

1. Tuples in Python

A tuple is a collection of items that is ordered and immutable (unchangeable). Tuples are similar to lists but once a tuple is created, you cannot modify it. They are often used to group related data together.

Syntax

```
my_tuple = (element1, element2, element3, ...)
```

Example:

```
my_tuple = ("apple", "banana", "cherry")
```

```
numbers_tuple = (1, 2, 3, 4)
```

Creating a Tuple with one element:

To create a tuple with only one element, include a trailing comma.

```
single_element_tuple = ("apple",)
```

2. Accessing Tuple Elements

You can access elements in a tuple using indexing, just like with lists. Tuples also support negative indexing.

Example:

```
fruits = ("apple", "banana", "cherry")
```

```
print(fruits[0]) # output: apple
```

```
print(fruits[-1]) # output: cherry
```


Slicing Tuples:

You can also slice tuples to access a subset of the elements.
`print(fruits[1:3])` # output ('banana', 'cherry')

Tuple operations

You can combine two or more tuples using the `+` operator.

`tuple1 = (1, 2, 3)`

`tuple2 = (4, 5, 6)`

`combined_tuple = tuple1 + tuple2`

`print(combined_tuple)` # output: (1, 2, 3, 4, 5, 6)

Tuple Repetition:

You can repeat a tuple multiple times using the `*` operator.

`repeated_tuple = (1, 2) * 3`

`print(repeated_tuple)` # output: (1, 2, 1, 2, 1, 2)

Checking membership:

You can check if an item exists in a tuple using the `in` operator.

`print("apple" in fruits)` # output: True

④ Tuple Methods

Though tuples are immutable, Python provides some built-in methods for working with tuples.

- `count()`: Return the number of times an element appears in the tuple.

`my_tuple = (1, 2, 3, 1, 1)`

`print(my_tuple.count(1))` # output: 3

- `index()`: Return the index of the first occurrence of an element in the tuple.

`my_tuple = ("apple", "banana", "cherry")`

`print(my_tuple.index("banana"))` # output: 1

• `index()`: Returns the index of the first occurrence of an element

`my_tuple = ("apple", "banana", "cherry")`

`Print(my_tuple.index("banana"))` # output: 1.

5. Advantages of using Tuples

- **Immutable**: This property ensures that tuple data cannot be modified creation, making them useful for fixed data.
- **Faster than Lists**: Due to immutability, tuples are generally faster than lists.
- **can be used as keys in dictionaries**: Since tuples are hashable, they can be used as keys in dictionaries unlike lists.

6. Set in Python

A set is a collection of unique items that is unordered and unindexed. Set do not allow duplicate values. Sets are useful for performing operations like union, intersection, and difference.

Syntax:

`my_set = {element1, element2, element3, ...}`

Example:

`fruits_set = {"apple", "banana", "cherry"}`

`numbers_set = {1, 2, 3, 4, 5}`

Empty Set:

To create an empty set, use the `set()` function (not `{}`, which creates an empty dictionary).

`empty_set = set()`

⑦ Set operations: Set support mathematical operations like union, intersection, and difference.

Union:

The union of two sets combines all elements from both sets, removing duplicates.

Set 1 = {1, 2, 3}

Set 2 = {3, 4, 5}

union-set = set 1 | set 2

output: {1, 2, 3, 4, 5}

Intersection

The intersection of two sets returns elements that are common to both sets.

intersection-set 1 & set 2 # output: {3}

Difference:

The difference between two sets returns elements that are in the first set but not in the second.

difference-set = set 1 - set 2 # output: {1, 2}

Symmetric Difference:

The symmetric difference returns elements that are in either of the sets, but not in both.

Sym-diff-set = set 1 ^ set 2 # output: {1, 2, 4, 5}

⑧ Set methods

Sets come with several useful methods for performing common

- add(): Adds an element to the set

fruits-set.add("orange")

Print(fruits-set) # output: {'apple', 'banana', 'cherry', 'orange'}

- `remove()`: Removes a specified element from the set. Raises error if the element does not exist.

```
fruits-set.remove("banana")
print(fruits-set) # output: {'apple', 'cherry'}
```

- `discard()`: Removes a specified element without raising error if it does not exist.

```
fruits-set.discard("banana") # no error if "banana" is not in the set
```

- `pop()`: Removes a random element from the set.

```
fruits-set.pop()
```

- `clear()`: Removes all elements from the set.

```
fruits-set.clear()
```

⑨ Differences Between Lists, Tuples, and sets

Feature	List	Tuple	Set
Ordering	Ordered	Ordered	Unordered
Mutability	Mutable	Immutable	Mutable
Duplicates	Allows duplicates	Allow duplicates	No duplicates
Indexing	Supports indexing	Supports indexing	No indexing
Operations	List operations	Tuple operations	Set operations
Common Uses	General collection	Fixed data	Unique items

Dictionaries in Python

A dictionary in Python is a collection of Key: value Pairs. Each Key in dictionary is associated with a value, and you can retrieve or manipulate data using the key. Unlike lists and tuples, dictionaries are unordered and mutable (changeable).

1. Creating a Dictionary

You can create a dictionary using curly braces {} or the dict() function.

Syntax:

```
my_dict = {  
    "key1": "value1",  
    "key2": "value2",  
    "key3": "value3",  
}
```

Example:

Let's create a dictionary of famous cities in Karnataka and their popular dishes.

```
Karnataka-food = {  
    "Bengaluru": "Bisi Bele Bath",  
    "Mysuru": "Mysore Pak",  
    "Mangaluru": "Neer Dosa",  
}
```

Accessing Dictionary Elements

To access the values stored in a dictionary, you use the key.

Example

```
Print(Karnataka-food["mysuru"]) # output : Mysore Pak
```


You can also use the `get()` method to access values, which is safer because it doesn't throw an error if the key doesn't exist.

```
Print (Karnataka-food.get("mangaluru")) # output: neer dosa
Print (Karnataka-food.get("shivamogga", "not found"))
# output: not found.
```

3. Adding and updating Dictionary Elements

You can add new key-value pairs or update existing values in a dictionary.

Adding an Item:

```
Karnataka-food["shivamogga"] = "Kadubu"
Print (Karnataka-food)
```

Updating an Item:

```
Karnataka-food["Bengaluru"] = "Raji modde"
```

4. Removing Elements from a Dictionary

You can remove items from a dictionary using several methods.

- `pop()`: Removes the specified key and returns the associated value.

```
mysuru-food = Karnataka-food.pop("mysuru")
```

```
Print (mysuru-food) # output: mysore Park
```

- `del`: Removes the specified key.

```
del Karnataka-food["mangaluru"]
```

- `clear()`: Empties the dictionary.

```
Karnataka-food.clear()
```


5

Dictionary methods

Here are some common methods available for dictionaries:

- **keys()**: Returns all the keys in the dictionary.
 Print(Karnataka - food.keys()) # output: dict_keys(['Bengaluru', 'mysuru', 'mangaluru'])
- **values()**: Returns all the values in the dictionary.
 Print(Karnataka - food.values()) # output: dict_values(['Bisi Balle', ('mysuru', 'mysore Pak'), ('mangaluru', 'neer dosa')])
- **update()**: Updates the dictionary with another dictionary or iterable.
 new_dishes = {"Hubballi": "Girmit"}
 Karnataka - food.update(new_dishes)

6 Dictionary characteristics

- **unordered**: Dictionary keys are not stored in any particular order.
- **Mutable**: You can change, add, or remove items.
- **Keys must be Immutable**: Keys in a dictionary must be of a data type that is immutable, such as a string, number, or tuple.
- **Unique Keys**: A dictionary cannot have duplicate keys. If you try to add a duplicate key, the latest value will overwrite the previous one.

7 Difference Between Lists, Tuples, Sets, and Dictionaries.

Feature	List	Tuple	Set	Dictionary
Ordering	ordered	ordered	unordered	unordered
Mutability	mutable	Immutable	mutable	mutable
Duplicates	Allows duplicates	Allows duplicate	no duplicates	key, no duplicate
Indexing	supports indexing	supports indexing	no indexing	uses keys
Structure	Indexed collection	Indexed collection	unordered collection	key-value pair